

Build Prompt: Muster & Wages

Copy everything below into Antigravity as your build instruction.

Build a full-stack web app called **Muster & Wages** — an attendance and payroll tracker for daily-wage labour teams (construction crews, factory floor workers, contract staff). It must be a real full-stack app with a persistent database and a backend API, not a client-only/localStorage prototype.

Tech Stack

- **Frontend:** React (Vite), React Router for the 3 tabs, fetch/axios for API calls. Mobile-first responsive CSS (plain CSS or CSS modules — no heavy UI kit, since the design is custom/industrial, not generic SaaS).
- **Backend:** Node.js + Express, REST API.
- **Database:** SQLite via Prisma ORM (simple, file-based, survives across sessions; can be swapped for Postgres later by changing the Prisma datasource).
- **Structure:** monorepo with `/client` and `/server` folders, or a single Express server that also serves the built React app in production.

Data Model (Prisma schema)

```
model Employee {
  id          String    @id @default(cuid())
  name        String
  role        String
  dailyRate   Float
  createdAt   DateTime  @default(now())
  attendance  AttendanceRecord[]
}

model AttendanceRecord {
  id          String    @id @default(cuid())
  employeeId  String
  employee    Employee  @relation(fields: [employeeId], references: [id], onDelete:
  date        String    // YYYY-MM-DD
```

```

    status      String    // "present" | "half" | "absent"

    @@unique([employeeId, date])
}

```

Backend API

- GET /api/employees — list all
- POST /api/employees — create { name, role, dailyRate }
- PUT /api/employees/:id — update
- DELETE /api/employees/:id — delete (cascades attendance records)
- GET /api/attendance?date=YYYY-MM-DD — attendance for all employees on a date
- PUT /api/attendance — upsert { employeeId, date, status } (status can be null to clear back to blank — delete the record in that case)
- GET /api/payroll?month=YYYY-MM — returns per-employee aggregates for the month: presentDays, halfDays, absentDays, paidDays, grossPay, plus a grand total

Core Features

1. Worker Roster tab

- Add / edit / delete workers: full name, role/trade (e.g. Mason, Electrician), daily wage rate in ₹ (INR).
- List view showing each worker's name, role, and rate/day.
- Deleting a worker should ask for confirmation (it also removes their attendance history via cascade).

2. Daily Attendance tab

- Date picker with prev/next day arrow navigation to select the day being marked.
- For every worker in the roster, a tappable status control (styled like a punch-card stamp) that cycles on tap: **blank** → **Present** → **Half Day** → **Absent** → **blank**.
- Each tap immediately persists via PUT /api/attendance (optimistic UI update, then confirm/rollback on failure).
- Attendance is stored per worker per date; loading a date should reflect whatever's

already saved.

3. Monthly Payroll tab

- Month picker to select the pay period.
- Auto-calculated per worker: count of Present days, count of Half days, count of Absent days, total Paid Days (Present + Half $\times$ 0.5), Gross Pay (Paid Days $\times$ daily rate).
- A grand total payout row summing all workers.
- "Export CSV" button that downloads the payroll table for the selected month (name, role, rate, present, half, absent, paidDays, grossPay).
- Table rendered on a light "paper ledger" surface, visually distinct from the dark app shell.

Design System

This should feel like an industrial workshop tool / physical wage ledger, not a generic SaaS dashboard.

- App shell background: dark charcoal #1c1f1e
- Panel/card surfaces: #24292a
- Accent color (buttons, active states, highlights): safety amber #f2a900
- Attendance status colors: Present = moss green, Half Day = amber, Absent = rust red #c24a3b
- Payroll table surface: light paper ledger tone #ece7db , used only for the payroll table itself to contrast against the dark shell — like a physical timesheet
- Typography:
 - Headers/labels: condensed uppercase display face — **Oswald** (Google Fonts)
 - Body text: **Inter**
 - All numbers, currency, and dates: monospace — **IBM Plex Mono** (gives it a ledger/timesheet feel)
- Attendance status buttons: styled like a punch-card stamp — dashed border, and a small "stamp-in" animation (quick scale + opacity pulse) on tap/click
- Layout: 3 primary tabs (Roster / Attendance / Payroll) — on mobile these should be a bottom tab bar (thumb-reachable, job-site usage on phones); on wider screens a top

nav is fine

- Mobile-first: assume this is used one-handed on a phone at a job site — large tap targets, minimal typing, date/month pickers that are easy to use on touch screens

Nice-to-Have (implement only if core is solid and time allows)

- Simple login/auth (single shared password or per-supervisor accounts) so multiple supervisors can use it
- Additional leave types beyond present/half/absent (e.g. paid leave, holiday)
- Optional overtime hours field per worker per day
- Multi-month payroll history view (not just the currently selected month)

Deliverable

A working full-stack app I can run locally with clear setup instructions (install deps, run migrations/seed, start dev servers for client + server). Seed the database with 4-5 sample workers with different roles and rates so the UI isn't empty on first load.